Risk Assessment and Mitigation Plan: Real-Time IoT Data Pipeline

1. Purpose

This document identifies the main risks that could affect the project, rates them, and defines how the team prevents them or responds if they happen.

2. Rating Method

- **Likelihood:** Low (1), Medium (2), High (3)
- **Impact:** Low (1), Medium (2), High (3)
- **Risk Score** = Likelihood x Impact. A score of 6 to 9 is high priority, 3 to 4 is medium, and 1 to 2 is low.

3. Risk Register

ID	Risk	Category	Likelihood	Impact	Score	Owner
R1	Team members are new to Kafka, Snowflake, or streaming, which slows progress	Technical / Skills	3	2	6	Rana
R2	Kafka setup or configuration problems (broker not starting, topics not reachable)	Technical	2	3	6	Waleed
R3	Data schema changes after components are built, breaking other members' parts	Integration	2	3	6	Mostafa
R4	Dependencies between members cause delays (one part waits for another)	Schedule	3	2	6	Rana
R5	Data loss or duplicate messages between producer, consumer, and warehouse	Data Quality	2	3	6	Abdelrahman
R6	Snowflake access problems or trial account limits	Infrastructure	2	2	4	Shahd

ID	Risk	Category	Likelihood	Impact	Score	Owner
R7	Simulated data is unrealistic or does not trigger anomalies for testing	Data Quality	2	2	4	Mostafa
R8	Streaming pipeline slows down or lags under continuous data flow	Performance	2	2	4	Abdelrahman
R9	Alerts are too frequent (false positives) or missed (false negatives)	Technical	2	2	4	Mostafa
R10	Dashboard cannot connect to Snowflake or the alert stream in real time	Technical	2	3	6	Rana
R11	Code or documents are lost or overwritten because of poor version control	Process	2	3	6	Rana
R12	A team member is unavailable for part of the project	Team	2	2	4	Rana
R13	Documentation and report are left to the last week	Schedule	3	2	6	Rana
R14	Local machines lack resources to run Kafka, the API, and the simulator together	Infrastructure	2	2	4	All

4. Mitigation and Contingency Plan

ID	Mitigation (Prevention)	Contingency (If It Happens)
R1	Share tutorials and reference repositories in Week 1; pair members on first tasks; ask questions early	Simplify the task (for example, a simpler consumer first) and have the owner pair with a teammate
R2	Use a tested Docker setup for Kafka; document the setup steps and share the configuration	Fall back to file-based ingestion (allowed by the proposal) until Kafka is fixed
R3	Agree and freeze the schema in Week 1; any change must be approved by the team leader and announced	Update all affected components together in one planned fix session

ID	Mitigation (Prevention)	Contingency (If It Happens)
R4	Agree on interfaces early; use sample files and mock data so members can work in parallel	Team leader re-balances tasks and helps unblock the delayed member
R5	Add unique message IDs, use consumer offsets correctly, and compare record counts at each stage	Reload from batched files and remove duplicates using message ID
R6	Create the Snowflake account early in Week 1 and test the connection	Use PostgreSQL or a local SQL database as a temporary warehouse
R7	Use realistic value ranges with small random variation; add a way to inject anomalies on purpose	Adjust the simulator parameters and re-run the tests
R8	Test with continuous flow early; keep processing logic simple and efficient	Reduce processing load or batch size and optimize the consumer
R9	Review thresholds with the team; test with normal and abnormal data	Tune thresholds and add a small tolerance or repeat-count check
R10	Test the dashboard connections as soon as the first data is available (Week 2)	Read from exported files or a local table instead of a live connection
R11	Use GitHub from day one with a clear folder structure, branches, and regular commits	Restore from commit history
R12	Each component has a README so others can continue; keep tasks visible on a shared board	Team leader redistributes the tasks
R13	Each member documents their own part during the week it is built; the report is compiled in Week 4	Assign remaining sections and hold a dedicated documentation session
R14	Check resource use early; stop unused services	Run components in turns or on another team member's machine

5. Risk Monitoring

- Risks are reviewed at each weekly meeting.

- Any new risk is added to the register with an owner, rating, and mitigation.
- The team leader escalates high-priority risks (score 6 or more) to the lecturer if the team cannot resolve them.

6. Top Priority Risks

The risks with the highest score and the most impact on the project are: team skills with new tools (R1), Kafka setup (R2), schema changes (R3), dependencies between members (R4), and data loss between stages (R5). These get early attention in Weeks 1 and 2.